COA- UNIT -1

Introduction:

A computer is an electronic device that manipulates information or data. It has the ability to store, retrieve, and process data.

1. **FIRST GENERATION**
    1. 1946-1959 is the period of first generation computer.
    2. Program and data reside in the same memory (stored program concepts –John von Neumann)
    3. ALP was made used to write programs
    4. Vacuum tubes were used to implement the functions (ALU & CU design)
    5. Magnetic core and magnetic tape storage devices are used
    6. Using electronic vacuum tubes, as the switching components
    7. J.P.Eckert and J.W.Mauchy invented the first successful electronic computer called ENIAC, ENIAC stands for "Electronic Numeric Integrated And Calculator".

- **Few Examples are:**
    1. ENIAC, EDVAC, UNIVAC, IBM-701, IBM-650
- **Advantages:**
    1. It made use of vacuum tubes which are the only electronic component available during those days.
    2. These computers could calculate in milliseconds.
- **Disadvantages:**
    1. These were very big in size, weight was about 30 tones.
    2. These computers were based on vacuum tubes.
    3. These computers were very costly.
    4. It could store only a small amount of information due to the presence of magnetic drums.
    5. As the invention of first generation computers involves vacuum tubes, so another disadvantage of these computers was, vacuum tubes require a large cooling system.
    6. Very less work efficiency.
    7. Limited programming capabilities and punch cards were used to take inputs.
    8. Large amount of energy consumption.
    9. Not reliable and constant maintenance is required.

2. **SECOND GENERATION**

- **Introduction:**
    1. 1959-1965 is the period of second-generation computer.
    2. Second generation computers were based on Transistor instead of vacuum tubes.
    3. Transistor were used to design ALU & CU
    4. HLL is used (FORTRAN)
    5. To convert HLL to MLL compiler were used
    6. Separate I/O processor were developed to operate in parallel with CPU, thus improving the performance

7. Invention of the transistor which was faster, smaller and required considerably less power to operate

- **Few Examples are:**

1. Honeywell 400, IBM 7094, CDC 1604, CDC 3600

- **Advantages:**
  1. Due to the presence of transistors instead of vacuum tubes, the size of electron component decreased. This resulted in reducing the size of a computer as compared to first generation computers.
  2. Less energy and not produce as much heat as the first genration.
  3. Assembly language and punch cards were used for input.
  4. Low cost than first generation computers.
  5. Better speed, calculate data in microseconds.
  6. Better portability as compared to first generation
- **Disadvantages:**
  1. A cooling system was required.
  2. Constant maintenance was required.
  3. Only used for specific purposes.

3. THIRD GENERATION

- **Introduction:**
  1. 1965-1971 is the period of third generation computer.
  2. These computers were based on Integrated circuits.
  3. IC was invented by Robert Noyce and Jack Kilby In 1958-1959.
  4. IC was a single component containing number of transistors.
  5. IC technology improved
  6. Improved IC technology helped in designing low cost, high speed processor and memory modules
  7. Multiprogramming, pipelining concepts were incorporated
  8. DOS allowed efficient and coordinate operation of computer system with multiple users
  9. Cache and virtual memory concepts were developed
  10. More than one circuit on a single silicon chip became available
- **Few Examples are:**

1. PDP-8, PDP-11, ICL 2900, IBM 360

- **Advantages:**
  1. These computers were cheaper as compared to second-generation computers.
  2. They were fast and reliable.
  3. Use of IC in the computer provides the small size of the computer.
  4. IC not only reduce the size of the computer but it also improves the performance of the computer as compared to previous computers.
  5. This generation of computers has big storage capacity.
  6. Instead of punch cards, mouse and keyboard are used for input.
  7. They used an operating system for better resource management and used the concept of time-sharing and multiple programming.

8. These computers reduce the computational time from microseconds to nanoseconds.

- **Disadvantages:**
  1. IC chips are difficult to maintain.
  2. The highly sophisticated technology required for the manufacturing of IC chips.
  3. Air conditioning is required.

4. FOURTH GENERATION

- **Introduction:**
  1. 1971-1980 is the period of fourth generation computer.
  2. This technology is based on Microprocessor.
  3. A microprocessor is used in a computer for any logical and arithmetic function to be performed in any program.
  4. Graphics User Interface (GUI) technology was exploited to offer more comfort to users.
- **Few Examples are:**
  1. IBM 4341
- **Advantages:**
  1. Fastest in computation and size get reduced as compared to the previous generation of computer.
  2. Heat generated is negligible.
  3. Small in size as compared to previous generation computers.
  4. Less maintenance is required.
  5. All types of high-level language can be used in this type of computers.
- **Disadvantages:**
  1. The Microprocessor design and fabrication are very complex.
  2. Air conditioning is required in many cases due to the presence of ICs.
  3. Advance technology is required to make the ICs.

5. FIFTH GENERATION

- **Introduction:**
  1. The period of the fifth generation in 1980-onwards.
  2. This generation is based on artificial intelligence.
  3. The aim of the fifth generation is to make a device which could respond to natural language input and are capable of learning and self-organization.
  4. This generation is based on ULSI(Ultra Large Scale Integration) technology resulting in the production of microprocessor chips having ten million electronic component.
- **Few Examples are:**
  1. Desktop, Laptop, NoteBook, UltraBook
- **Advantages:**
  1. It is more reliable and works faster.
  2. It is available in different sizes and unique features.
  3. It provides computers with more user-friendly interfaces with multimedia features.
- **Disadvantages:**
  1. They need very low-level languages.
  2. They may make the human brains dull and doomed.

Computer Architecture:

Computer Architecture refers to those attributes of a system that have a direct impact on the logical execution of a program. Examples:

- the instruction set
- the number of bits used to represent various data types
- I/O mechanisms
- memory addressing techniques

Computer Organization:

Computer Organization refers to the operational units and their interconnections that realize the architectural specifications. Examples are things that are transparent to the programmer:

- control signals
- interfaces between computer and peripherals
- the memory technology being used

Organization is nothing but how features are implemented

Example:

It is an Architectural design issue that - whether a computer will have a multiply instruction.

And it is an Organization issue that - whether that instruction(Multiply Instruction) will be implemented by special hardware of Multiply unit or by a mechanism that make repeated use of add unit of the system.

Many computer manufacturers offer a family of computer models, all with the same Architecture but with differences in Organization like some computer models have more cache, some have less, speed of some computer is fast, while some have slow, but architecture that is, computer have one cache, one hard disk etc are same, in all models.

Structure and Function

Structure is the way in which components relate to each other

Four main structural components:

- Central processing unit (CPU)
- Main memory
- I / O
- System interconnections

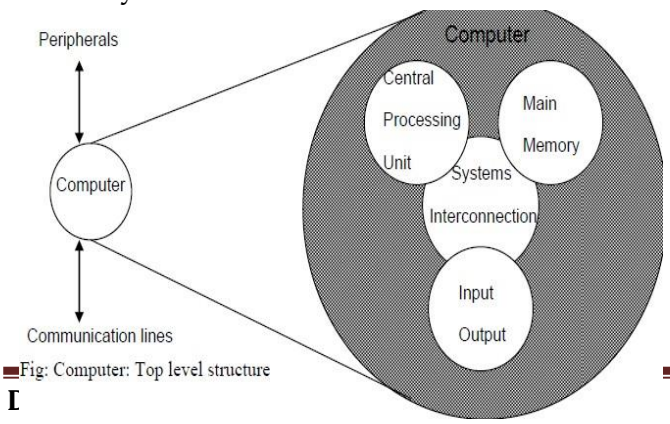


Fig: Computer: Top level structure

- CPU structural components:
  - Control unit
  - Arithmetic and logic unit (ALU)
  - Registers
  - CPU interconnections

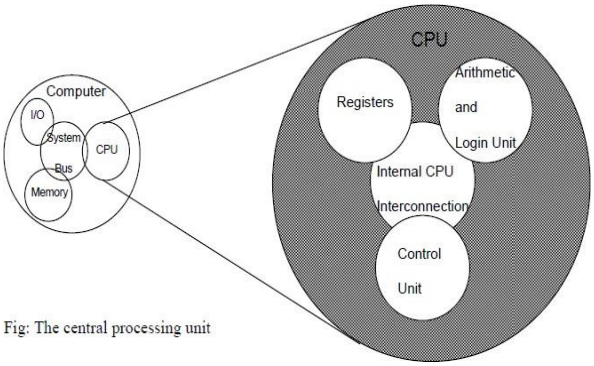


Fig: The central processing unit

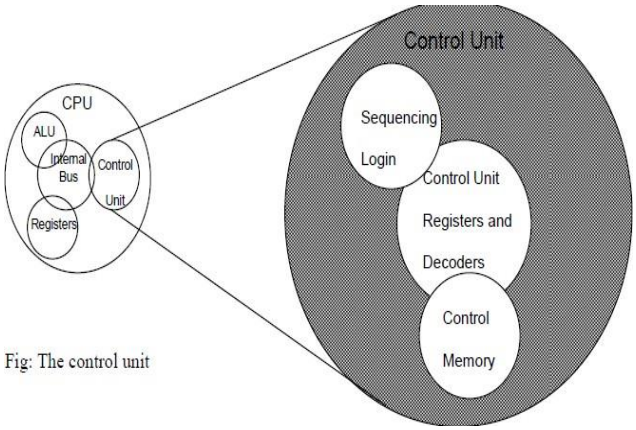


Fig: The control unit

Function is the operation of individual components as part of the structure

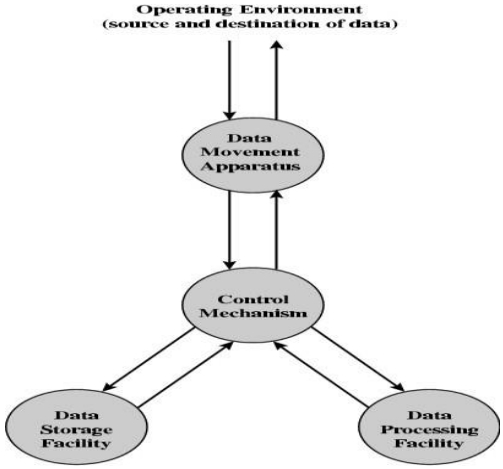
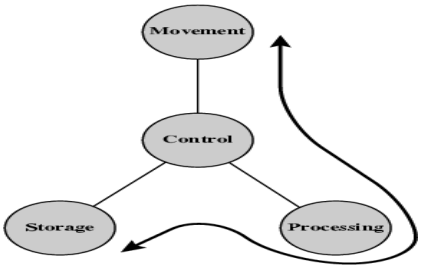


Fig: Functional view of Computer

All computer functions are:

1. **Data processing**: Computer must be able to process data which may take a wide variety of forms and the range of processing.



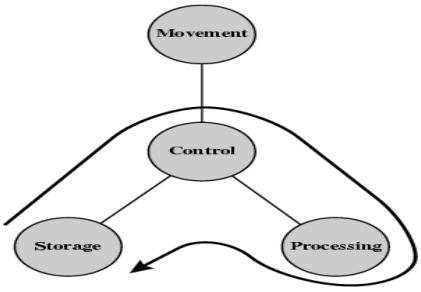
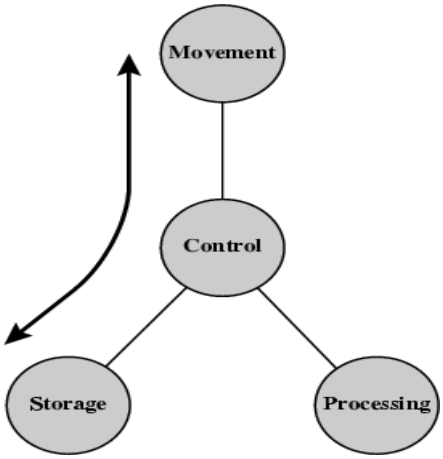
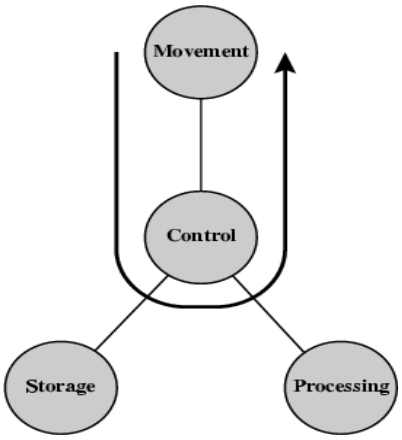


Fig: Processing from storage to I/O Fig: Data processing from / to Storage

2. **Data storage**: Computer stores data either temporarily or permanently.



3. **Data movement**: Computer must be able to move data between itself and the outside world.



4. **Control**: There must be a control of the above three functions.

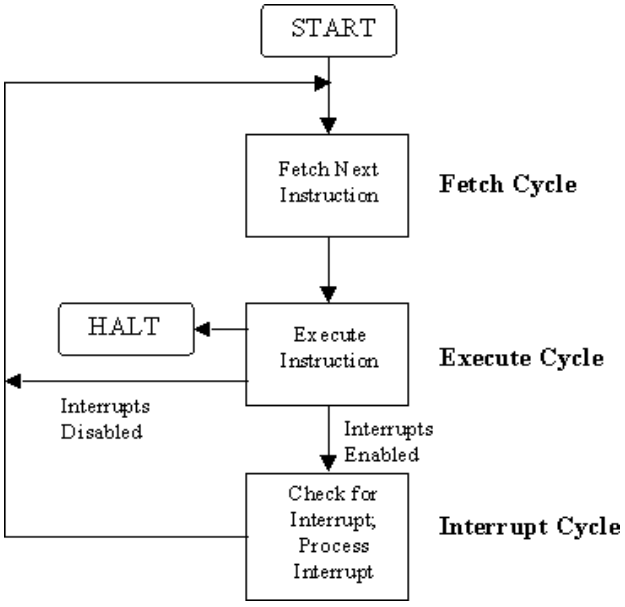
Instruction Execution Cycle

The basic function performed by a computer is execution of a program, which consists of a set of instructions stored in memory. The processor does the actual work by executing instructions specified in the program.

It mainly consists of two phases:

1. Fetch
2. Execute
3. Interrupt

In an improved instruction execution cycle, we can introduce a third cycle known as the **interrupt cycle**. Below figure shows how the interrupt cycle fits into the overall cycle.



Instruction cycle with Interrupts

Accumulator (ACC)

A register located on the central processing unit. The contents can be used by the arithmetic-logic unit for arithmetic and logic operations, and by the memory buffer register. Usually, all results generated by the arithmetic-logic unit end up in the accumulator.

Arithmetic-Logic Unit (ALU)

Performs arithmetic operations such as addition and subtraction as well as logical operations such as AND, OR and NOT. Most operations require two operands. One of these operands usually comes from memory via the memory buffer register, while the other is the previously loaded value stored in the accumulator. The results of an arithmetic logic unit operation is usually transferred to the accumulator.

Clock Pulse

A circuit which sends pulses of current to the central processing unit. Nothing happens in the computer until a pulse is received. The measurement of the number of clock pulses sent every second is expressed in megahertz (MHz), for example, a 100MHz computer sends 100 million pulses every second.

Control Unit (CU)

Circuitry located on the central processing unit which coordinates and controls all hardware. This is accomplished by using the contents of the instruction register to decide which circuits are to be activated.

Instruction Register (IR)

A register located on the central processing unit which holds the contents of the last instruction fetched. This instruction is now ready to be executed and is accessed by the control unit.

Memory

Memory is made up of a series of zero's (0) and one's (1) called bits or binary. These individual bits are grouped together in lots of eight and are referred to as a byte. Every byte in memory can be accessed by a unique address which identifies its location. The memory in modern computers contains millions of bytes and is often referred to as random access memory (RAM).

Memory Address Register (MAR)

A register located on the central processing unit which is in turn connected to the address lines of the system. This register specifies the address in memory where information can be found and can be also used to point to a memory location where information is to be stored.

Memory Buffer Register (MBR)

A register located on the central processing unit which is in turn connected to the data lines of the system. The main purpose of this register is to act as an interface between the central processing unit and memory. When the appropriate signal is received by the control unit, the memory location stored in the memory address register is used to copy data from or to the memory buffer register.

Program Counter (PC)

PC contains the memory address of the next instruction to be executed. The contents of the program counter are copied to the memory address register before an instruction is fetched from memory. At the completion of the fetched instruction, the control unit updates the program counter to point to the next instruction which is to be fetched.

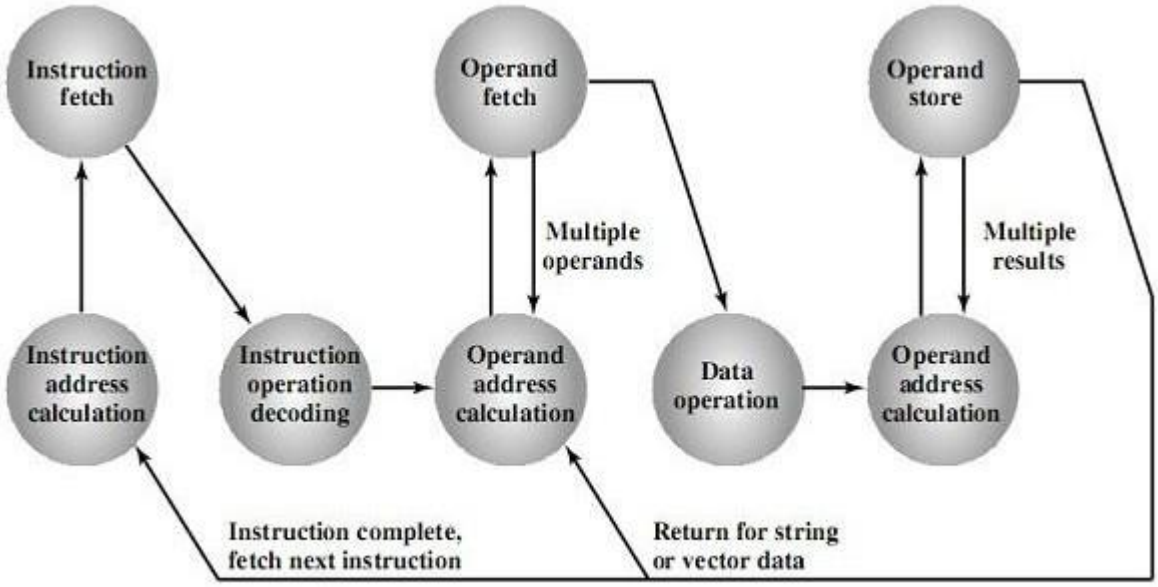
Instruction Set Architecture:

ISA is the structure of a computer that a machine language programmer must understand to write a correct program for that machine.

The ***Instruction Set Architecture*** (ISA) is the part of the processor that is visible to the programmer or compiler writer. The ISA serves as the boundary between software and hardware.

Elements of Machine Instructions:

The operation of the processor is determined by the instructions it executes, referred to as machine instructions or computer instructions. The collection of different instructions that the processor can execute is referred to as the processor‟s instruction set.



Instruction Cycle State Diagram

- **Instruction address calculation (iac):** Determine the address of the next instruction to be executed. Usually, this involves adding a fixed number to the address of the previous instruction. For example, if each instruction is 16 bits long and memory is organized into 16-bit words, then add 1 to the previous address. If, instead, memory is organized as individually addressable 8-bit bytes, then add 2 to the previous address.
- **Instruction fetch (if):** Read instruction from its memory location into the processor.
- **Instruction operation decoding (iod):** Analyze instruction to determine type of operation to be performed and operand(s) to be used.
- **Operand address calculation (oac):** If the operation involves reference to an operand in memory or available via I/O, then determine the address of the operand.
- **Operand fetch (of):** Fetch the operand from memory or read it in from I/O.
- **Data operation (do):** Perform the operation indicated in the instruction.

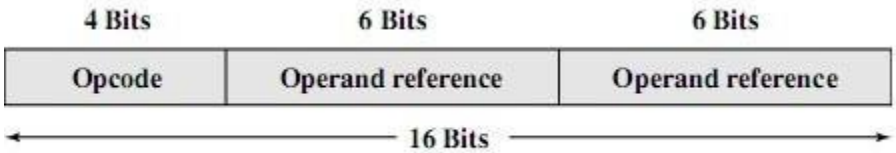
Elements of a Machine Instruction:

Each instruction must contain the information required by the processor for execution. Above figure shows the steps involved in instruction execution and, by implication, defines the elements of a machine instruction.

These elements are as follows:

- **Operation code:** Specifies the operation to be performed (e.g., ADD, I/O). The operation is specified by a binary code, known as the operation code, or opcode.
- **Source operand reference:** The operation may involve one or more source operands, that is, operands that are inputs for the operation.
- **Result operand reference:** The operation may produce a result.
- **Next instruction reference:** This tells the processor where to fetch the next instruction after the execution of this instruction is complete.

The address of the next instruction to be fetched could be either a real address or a virtual address, depending on the architecture. Generally, the distinction is transparent to the instruction set architecture. In most cases, the next instruction to be fetched immediately follows the current instruction. In those cases, there is no explicit reference to the next instruction. When an explicit reference is needed, then the main memory or virtual memory address must be supplied.



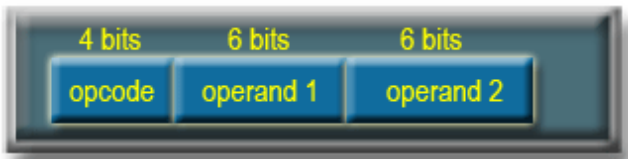
A Simple Instruction Format

Source and result operands can be in one of four areas:

i. **Main or virtual memory:** As with next instruction references, the main or virtual memory address must be supplied.
ii. **Processor register:** With rare exceptions, a processor contains one or more registers that may be referenced by machine instructions. If only one register exists, reference to it may be implicit. If more than one register exists, then each register is assigned a unique name or number, and the instruction must contain the number of the desired register.
iii. **Immediate:** The value of the operand is contained in a field in the instruction being executed.
iv. **I/O device:** The instruction must specify the I/O module and device for the operation. If memory-mapped I/O is used, this is just another main or virtual memory address.

Instruction representation:-

- Within the computer, each instruction is represented by a sequence of bits. The instruction is divided into fields, corresponding to the constituent elements of the instruction.
- Example: It is assume that it is a 16-bit CPU. 4 bits are used to provide the operation code. So, we may have to 16 (2^4= 16) different set of instructions. With each instruction, there are two operands. To specify each operands, 6 bits are used.



It is possible to provide 64 different operands for each operand reference. It is difficult to deal with binary representation of machine instructions. Thus, it has become common practice to use a symbolic representation of machine instructions.

Opcodes are represented by abbreviations, called mnemonics that indicate the operations.

Common examples include:

ADD	Add
SUB	Subtract
MULT	Multiply
DIV	Division
LOAD	Load data from memory to CPU
STORE	Store data to memory from CPU.

Operands are also represented symbolically. For example, the instruction may mean multiply the value contained in the data location X by the contents of register R and put the result in register R

MULT R, X ; R R * X

In this example, X refers to the address of a location in memory and R refers to a particular register.

Thus, it is possible to write a machine language program in symbolic form. Each symbolic opcode has a fixed binary representation, and the programmer specifies the location of each symbolic operand.

Instruction types:-

The instruction set of a CPU can be categorized as follows:

i. Data Processing
ii. Data Storage
iii. Data Movement
iv. Control

i. Data Processing:

Arithmetic and Logic instructions: Arithmetic instructions provide computational capabilities for processing numeric data. Logic (Boolean) instructions operate on the bits of a word as bits rather than as numbers. Logic instructions thus provide capabilities for processing any other type of data. There operations are performed primarily on data in CPU registers.

ii. Data Storage:

Memory instructions are used for moving data between memory and CPU registers.

iii. Data Movement:

I/O instructions are needed to transfer program and data into memory from storage device or input device and the results of computation back to the user.

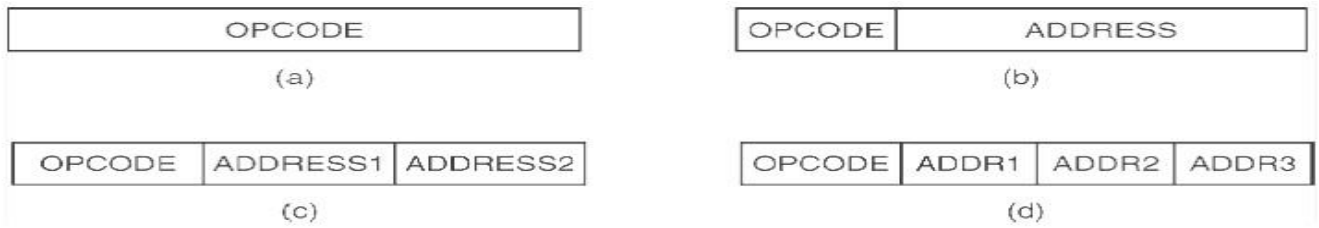
iv. Control:

Test and branch instructions Test instructions are used to test the value of a data word or the status of a computation. Branch instructions are then used to branch to a different set of instructions depending on the decision made.

Number of Addresses:

Most of the arithmetic and logic operations are either unary (one operand) or binary (two operands). Thus we need a maximum of two addresses to reference operands. The result of an operation must be stored, suggesting a third address. Finally after completion of an instruction, the next instruction must be fetched, and its address is needed.

This reasoning suggests that an instruction may require containing four address references: two operands, one result, and the address of the next instruction. In practice, four address instructions are rare. Most instructions have one, two or three operands addresses, with the address of the next instruction being implicit (obtained from the program counter).



Four common instruction formats:
(a) Zero-address instruction. (b) One-address instruction
(c) Two-address instruction. (d) Three-address instruction

Example:-

Using zero, one, two, or three address instruction. We will use the symbols ADD, SUB, MUL, and DIV for the four arithmetic operations; MOV for the transfer-type operation; and LOAD and STORE for transfers to and from memory and AC register. We will assume that the operands are in memory addresses A, B, C, and D, and the result must be stored in memory at address X.

THREE-ADDRESS INSTRUCTIONS

Computers with three-address instruction formats can use each address field to specify either a processor register or a memory operand. The program in assembly language that evaluates X = (A + B) * (C + D) is shown below, together with comments that explain the register transfer operation of each instruction.

ADD R1, A, B R1 ← M [A] + M [B]
ADD R2, C, D R2 ← M [C] + M [D]
MUL X, R1, R2 M [X] ← R1 * R2

It is assumed that the computer has two processor registers, R1 and R2. The symbol M [A] denotes the operand at memory address symbolized by A.

The advantage of the three-address format is that it results in short programs when evaluating arithmetic expressions. The disadvantage is that the binary-coded instructions require too many bits to specify three addresses. An example of a commercial computer that uses three-address instructions is the Cyber 170. The instruction formats in the Cyber computer are restricted to either three register address fields or two register address fields and one memory address field.

TWO-ADDRESS INSTRUCTIONS

Two address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word.

The program to evaluate X = (A + B) * (C + D) is as follows:

MOV R1, A	R1 ← M [A]
ADD R1, B	R1 ← R1 + M [B]
MOV R2, C	R2 ← M [C]
ADD R2, D	R2 ← R2 + M [D]
MUL R1, R2	R1 ← R1*R2
MOV X, R1	M [X] ← R1

The MOV instruction moves or transfers the operands to and from memory and processor registers. The first symbol listed in an instruction is assumed to be both a source and the destination where the result of the operation is transferred.

ONE-ADDRESS INSTRUCTIONS

One-address instructions use an implied accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register. However, here we will neglect the second and assume that the AC contains the result of tall operations. The program to evaluate X = (A + B) * (C + D) is

LOAD A	AC ← M [A]
ADD B	AC ← A [C] + M [B]
STORE T	M [T] ← AC
LOAD C	AC ← M [C]
ADD D	AC ← AC + M [D]
MUL T	AC ← AC * M [T]
STORE X	M [X] ← AC

All operations are done between the AC register and a memory operand. T is the address of a temporary memory location required for storing the intermediate result.

ZERO-ADDRESS INSTRUCTIONS

A stack-organized computer does not use an address field for the instructions ADD and MUL. The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack. The following program shows how X = (A + B) * (C + D) will be written for a stack organized computer. (TOS stands for top of stack)

PUSH A	TOS ← A
PUSH B	TOS ← B
ADD	TOS ← (A + B)
PUSH C	TOS ← C
PUSH D	TOS ← D
ADD	TOS ← (C + D)
MUL	TOS ← (C + D) * (A + B)
POP X	M [X] ← TOS

To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into reverse Polish notation. The name "zero-address" is given to this type of computer because of the absence of an address field in the computational instructions.

Example 2:

Instruction		Comment
SUB	Y, A, B	$Y \leftarrow A - B$
MPY	T, D, E	$T \leftarrow D \times E$
ADD	T, T, C	$T \leftarrow T + C$
DIV	Y, Y, T	$Y \leftarrow Y \div T$

(a) Three-address instructions

Instruction		Comment
MOVE	Y, A	$Y \leftarrow A$
SUB	Y, B	$Y \leftarrow Y - B$
MOVE	T, D	$T \leftarrow D$
MPY	T, E	$T \leftarrow T \times E$
ADD	T, C	$T \leftarrow T + C$
DIV	Y, T	$Y \leftarrow Y \div T$

(b) Two-address instructions

Instruction		Comment
LOAD	D	$AC \leftarrow D$
MPY	E	$AC \leftarrow AC \times E$
ADD	C	$AC \leftarrow AC + C$
STOR	Y	$Y \leftarrow AC$
LOAD	A	$AC \leftarrow A$
SUB	B	$AC \leftarrow AC - B$
DIV	Y	$AC \leftarrow AC \div Y$
STOR	Y	$Y \leftarrow AC$

(c) One-address instructions

Programs to Execute $Y = \dfrac{A - B}{C + (D \times E)}$

Instruction Set Design:-

One of the most interesting, and most analyzed, aspects of computer design is instruction set design. The instruction set defines the functions performed by the CPU. The instruction set is the programmer's means of controlling the CPU. Thus programmer requirements must be considered in designing the instruction set.

Most important and fundamental design issues:

	Operation repertoire	:	How many and which operations to provide, and how complex operations should be.
	Data Types	:	The various type of data upon which operations are performed.
	Instruction format	:	Instruction length (in bits), number of addresses, size of various fields and so on.
	Registers	:	Number of CPU registers that can be referenced by instructions and their use.
	Addressing	:	The mode or modes by which the address of an operand is specified.

Data Types:-

Machine instructions operate on data. The most important general categorized data as follows:

1. Addresses
2. Numbers
3. Characters
4. Logical Data

1. **Addresses:** It basically indicates the address of a memory location. Addresses are nothing but the unsigned integer, but treated in a special way to indicate the address of a memory location.

Address arithmetic is somewhat different from normal arithmatic and it is related to machine architecture.

2. **Numbers:** All machine languages include numeric data types. Numeric data are classified into two broad categories: integer or fixed point and floating point.

3. **Characters:** A common form of data is text or character strings. Since computer works with bits, so characters are represented by a sequence of bits. The most commonly used coding scheme is ASCII (American Standard Code for Information Interchange) code.

4. **Logical Data:** Normally each word or other addressable unit (byte, halfword, and so on) is treated as a single unit of data. It is sometime useful to consider an *n*-bit unit as consisting of *n* 1-bit items of data, each item having the value 0 or 1. When data are viewed this way, they are considered to be logical data. Generally 1 is treated as true and 0 is treated as false.

Types of Operations (or) Instruction Types:-

The number of different opcodes and their types varies widely from machine to machine. However, some general types of operations are found in most of the machine architecture. Those operations can be categorized as follows:

1. Data Transfer
2. Arithmetic
3. Logical
4. Conversion
5. Input Output [I/O]
6. System Control
7. Transfer Control

1. Data Transfer:

The most fundamental type of machine instruction is the data transfer instruction. The data transfer instruction must specify several things. First, the location of the source and destination operands must be specified. Each location could be memory, a register, or the top of the stack. Second, the length of data to be transferred must be indicated. Third, as with all instructions with operands, the mode of addressing for each operand must be specified.

The CPU has to perform several task to accomplish a data transfer operation. If both source and destination are registers, then the CPU simply causes data to be transferred from one register to another; this is an operation internal to the CPU.

If one or both operands are in memory, then the CPU must perform some or all of the following actions:

i. Calculate the memory address, based on the addressing mode.
ii. If the address refers to virtual memory, translate from virtual to actual memory address.
iii. Determine whether the addressed item is in cache.
iv. If not, issue a command to the memory module.

Commonly used data transfer operation:

Operation Name	Description

Move (Transfer)	Transfer word or block from source to destination
Store	Transfer word from processor to memory
Load (fetch)	Transfer word from memory to processor
Exchange	Swap contents of source and destination
Clear (reset)	Transfer word of 0s to destination
Set	Transfer word of 1s to destination
Push	Transfer word from source to top of stack
Pop	Transfer word from top of stack to destination

2. Arithmetic:-

Most machines provide the basic arithmetic operations like add, subtract, multiply, divide etc. These are invariably provided for signed integer (fixed-point) numbers. They are also available for floating point number.

The execution of an arithmetic operation may involve data transfer operation to provide the operands to the ALU input and to deliver the result of the ALU operation.

Commonly used data transfer operation:

Operation Name	Description
Add	Compute sum of two operands
Subtract	Compute difference of two operands
Multiply	Compute product of two operands
Divide	Compute quotient of two operands
Absolute	Replace operand by its absolute value
Negate	Change sign of operand
Increment	Add 1 to operand
Decrement	Subtract 1 from operand

3. Logical:

Most machines also provide a variety of operations for manipulating individual bits of a word or other addressable units.

Most commonly available logical operations are:

Operation Name	Description
AND	Performs the logical operation AND bitwise
OR	Performs the logical operation OR bitwise
NOT	Performs the logical operation NOT bitwise
Exclusive OR	Performs the specified logical operation Exculsive-OR bitwise
Test	Test specified condition; set flag(s) based on outcome
Compare	Make logical or arithmatic comparison Set flag(s) based on outcome
Set Control Variables	Class of instructions to set controls for protection purposes,

	interrupt handling, timer control etc.
Shift	Left (right) shift operand, introducing constant at end
Rotate	Left (right) shift operation, with wraparound end

4. Conversion:

Conversion instructions are those that change the format or operate on the format of data. An example is converting from decimal to binary.

5. Input / Output :

Input / Output instructions are used to transfer data between input/output devices and memory/CPU register.

Commonly available I/O operations are:

Operation Name	Description
Input (Read)	Transfer data from specified I/O port or device to destination (e.g., main memory or processor register)
Output (Write)	Transfer data from specified source to I/O port or device.
Start I/O	Transfer instructions to I/O processor to initiate I/O operation.
Test I/O	Transfer status information from I/O system to specified destination

6. System Control:

System control instructions are those which are used for system setting and it can be used only in privileged state. Typically, these instructions are reserved for the use of operating systems. For example, a system control instruction may read or alter the content of a control register. Another instruction may be to read or modify a storage protection key.

7. Transfer of Control:

In most of the cases, the next instruction to be performed is the one that immediately follows the current instruction in memory. Therefore, program counter helps us to get the next instruction. But sometimes it is required to change the sequence of instruction execution and for that instruction set should provide instructions to accomplish these tasks. For these instructions, the operation performed by the CPU is to upload the program counter to contain the address of some instruction in memory. The most common transfer-of-control operations found in instruction set are: **branch, skip and procedure call**.

7.(a). Branch Instruction

- A branch instruction, also called a jump instruction, has one of its operands as the address of the next instruction to be executed.
- Basically there are two types of branch instructions: Conditional Branch instruction and unconditional branch instruction.
- In case of unconditional branch instruction, the branch is made by updating the program counter to address specified in operand.

- In case of conditional branch instruction, the branch is made only if a certain condition is met. Otherwise, the next instruction in sequence is executed

On such a machine, there could be four different conditional branch instructions:

BRP X ; Branch to location X if result is positive

BRN X ; Branch to location X if result is negative

BRZ X ; Branch to location X is result is zero

BRO X ; Branch to location X if overflow occurs

BRE R1, R2, X ; Branch to X if contents of R1 = Contents of R2.

7. (b). Skip Instruction

Another common form of transfer-of-control instruction is the skip instruction. Generally, the skip implies that one instruction to be skipped; thus the implied address equals the address of the next instruction plus one instruction length. A typical example is the increment-and-skip-if-zero (ISZ) instruction. For example,

ISZ R1

This instruction will increment the value of the register R1. If the result of the increment is zero, then it will skip the next instruction.

7. (c). Procedure Call Instruction

A procedure is a self contained computer program that is incorporated into a large program. At any point in the program the procedure may be invoked, or called. The processor is instructed to go and execute the entire procedure and then return to the point from which the call took place.

There are three common places for storing the return address:

- Register
- Start of procedure
- Top of stack

Most commonly used transfer of control operation:

Operation Name	Description
Jump (branch)	Unconditional transfer, load PC with specific address
Jump conditional	Test specific condition; either load PC with specific address or do nothing, based on condition
Jump to subroutine	Place current program control information in known location; jump to specific address
Return	Replace contents of PC and other register from known location
Skip	Increment PC to skip next instruction
Skip Conditional	Test specified condition; either skip or do nothing based on condition
Halt	Stop program execution

Addressing Modes:

1. **The addressing modes in computer architecture actually define how an operand is chosen to execute an instruction**. It is the way that is used to identify the location of an operand which is specified in an instruction.
2. Whenever an instruction executes, it requires operands to be operated on. An instruction field consisting of opcode and operand. Where operand means the data and opcode means the instruction itself. In case of operations like addition or subtraction, they require two data. So, they are called binary instruction. On the other hand, the increment or decrement operations need only one data and are so called unary instruction.
3. The various addressing modes in computer architecture can be classified as below. We have some other addressing modes too, but these are the prime addressing modes in computer architecture.
4. The technique for specifying the address of the operands is known as **Addressing Mode**.
5. The address where operand is actually found is known as "**Effective Address**".

Notations:

A= Contents of an address field in the instruction
R= Contents of an address field in the instruction that refers to a register
EA= Effective Address (Actual address) of location containing the referenced operand.
(X)= Contents of memory location x or register X.

Types of Addressing Modes:

Various types of addressing modes are:

1. Implicit Addressing Mode
2. Immediate Addressing Modes
3. Direct Addressing Modes
4. Indirect Addressing Modes
5. Register Addressing Modes
6. Register Indirect Addressing Mode
7. Auto-Increment and Auto-Decrement Addressing Modes
8. Displacement Based Addressing Modes
9. Stack Addressing

1. **Implicit addressing mode :**

   The term implicit addressing mode means here we are not mentioning clearly in details that from where the instruction can get the operand. But by default, the instruction itself knows from where it is supposed to access the operand. For example, **CMA** stands for complement accumulator. The meaning of the **CMA** instruction is whatever the value present in the accumulator will be replaced by its 1"s complement.

   In this instruction **CMA** or along with this instruction, we are not mentioning any operand. So here it knows that the operand has to be accessed from the accumulator implicitly. This is known as implicit addressing modes.

2. **Immediate Addressing Modes:**

In Immediate Addressing Mode operand is specified in the instruction itself. In other words, an immediate mode instruction has an operand field rather than an address field, which contain actual operand to be used in conjunction with the operand specified in the instruction.



As an example: The Instruction:

MOVE 06 Move 06 to the accumulator

ADD 05 ADD 05 to the content of accumulator

This mode is very useful for initializing the register to a constant value.

3. **Direct Addressing Mode:**

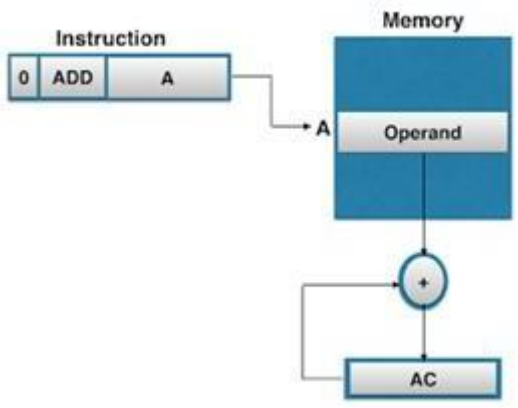
Direct Addressing Mode is also known as "Absolute Addressing Mode". In this mode the address of data (operand) is specified in the instruction itself. That is, in this type of mode, the operand resides in memory and its address is given directly by the address field of the instruction.

A very simple form of addressing is direct addressing, in which the address field contains the effective address of the operand:

i.e., EA=A

Ex:

ADD A Means add contents of cell A to accumulator .



Here, we see that in it Memory Address=Operand.

4. **Indirect Addressing Mode:**

In this mode, the address field of instruction gives the memory address where on, the operand is stored in memory. That is, in this mode, the address field of the instruction

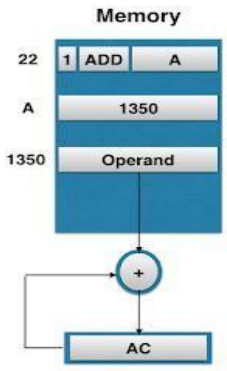
gives the address where the "Effective Address" is stored in memory.

i.e., EA=(A)

Means, here, Control fetches the instruction from memory and then uses its address part to access memory again to read Effective Address.

Ex:-

ADD (A) - Means adds the content of cell pointed to contents of A to Accumulator.



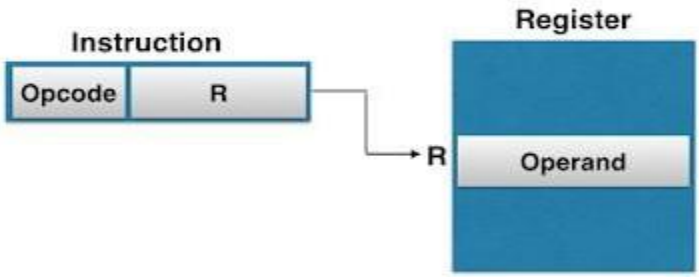
Thus in it, AC ← M [M[A]] [M=Memory]

i.e., (A)=1350=EA

5. **Register Addressing Mode:**

In Register Addressing Mode, the operands are in registers that reside within the CPU. That is, in this mode, instruction specifies a register in CPU, which contains the operand. It is like Direct Addressing Mode; the only difference is that the address field refers to a register instead of memory location.

i.e., EA=R



Examples of such instructions are:

MOV AX, BX Move contents of Register BX to AX

ADD AX, BX Add the contents of register BX to AX

Here, AX, BX are used as register names which is of 16-bit register.

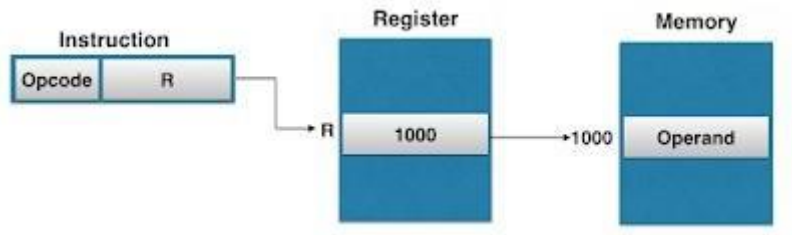
Thus, for a Register Addressing Mode, there is no need to compute the actual address as the operand is in a register and to get operand there is no memory access involved.

6. **Register Indirect Addressing Mode:**

In Register Indirect Addressing Mode, the instruction specifies a register in CPU whose contents give the operand in memory. In other words, the selected register contain the address of operand rather than the operand itself. That is,

i.e., EA=(R)

Means, control fetches instruction from memory and then uses its address to access Register and looks in Register(R) for effective address of operand in memory.



Here, the parentheses are to be interpreted as meaning contents of.

Ex:-

MOV AL, [BX]

Code example in Register:

MOV BX, 1000H

MOV 1000H, operand

From above example, it is clear that, the instruction(MOV AL, [BX]) specifies a register[BX], and in coding of register, we see that, when we move register [BX], the register contain the address of operand(1000H) rather than address itself.

7. **Auto-increment and Auto-decrement Addressing Modes:**

These are similar to Register indirect Addressing Mode except that the register is incremented or decremented after (or before) its value is used to access memory.

These modes are required because when the address stored in register refers to a table of data in memory, then it is necessary to increment or decrement the register after every access to table so that next value is accessed from memory. Thus, these addressing modes are common requirements in computer.

Auto-increment Addressing Mode:

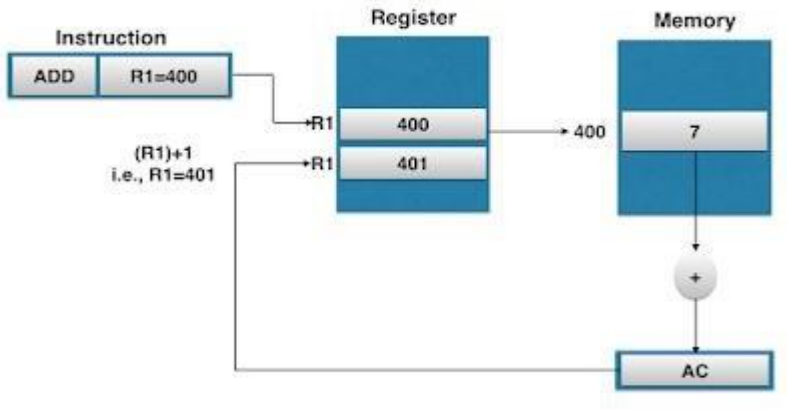
Auto-increment Addressing Mode is similar to Register Indirect Addressing Mode except that the register is incremented after its value is loaded (or accessed) at another location like accumulator (AC).

That is, in this case also, the Effective Address is equal to

EA=(R)

But, after accessing operand, register is incremented by 1.

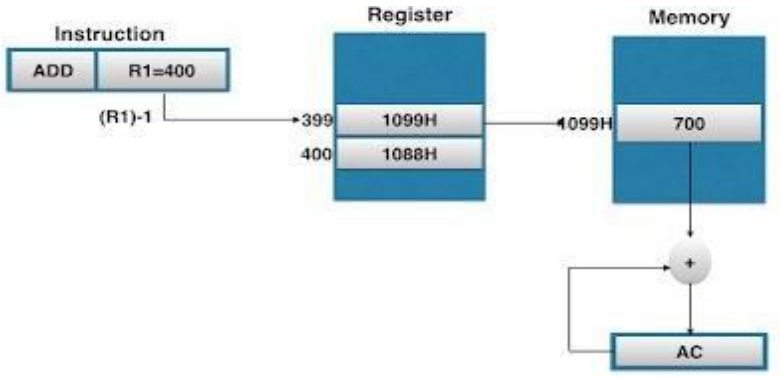
As an example:



Here, we see that effective address is (R)=400 and operand in AC is 7.
And after loading R1 is incremented by 1.It becomes 401.
Means, here we see that, in the Auto-increment mode, the R1 register is increment to 401 after execution of instruction.

Auto-decrement Addressing Mode:

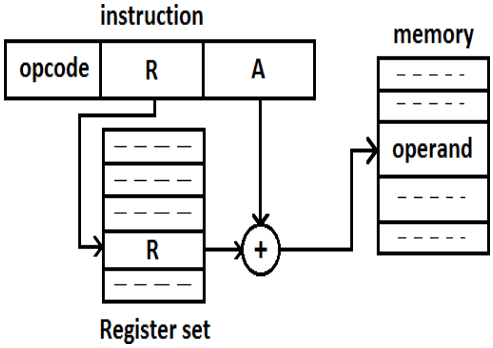
Auto-decrement Addressing Mode is reverse of auto-increment , as in it the register is decrement before the execution of the instruction. That is, in this case, effective address is equal to EA=(R) - 1



Here, we see that, in the Auto-decrement mode, the register R1 is decremented to 399 prior to execution of the instruction, means the operand is loaded to accumulator, is of address 1099H in memory, instead of 1088H.Thus, in this case effective address is 1099H and contents loaded into accumulator is 700.

8. **Displacement Based Addressing Modes**

In the displacement addressing mode, the instruction will be having three fields. One for the **opcode**, one for the **register number** and the remaining one for an **absolute address**.
i.e., EA=A+(R)
At first, depending upon the register number the register will be selected from the register set. After that its content will be added with the absolute address and the new address formed will be the actual physical address of the operand in the memory.



Displacement addressing mode in computer architecture can be categorized into 3 different modes.

a. **Relative**
b. **Base register**
c. **Indexing**

In case of relative addressing mode, the register used will be a **program counter**.

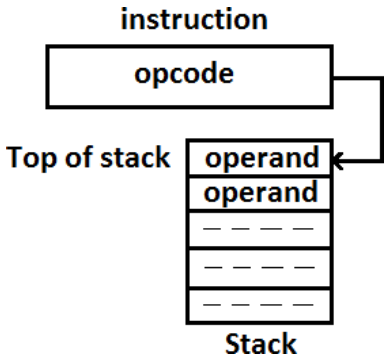
In the base addressing mode, the register will contain the **base address** and the absolute field will be the **offset or displacement** from the base address. After adding both the actual physical address of the operand can be obtained and mapping this address in the memory we can access the operand.

For example, if the base address is **3000** and the offset is **20**, then after adding both i.e. **3020** will be the actual address of the operand.

In case of Indexing mode, the absolute field will contain the **starting base address** of the memory block and the register field will contain the **index value**. Adding both will give the actual physical address of the operand.

9. **Stack Addressing:**

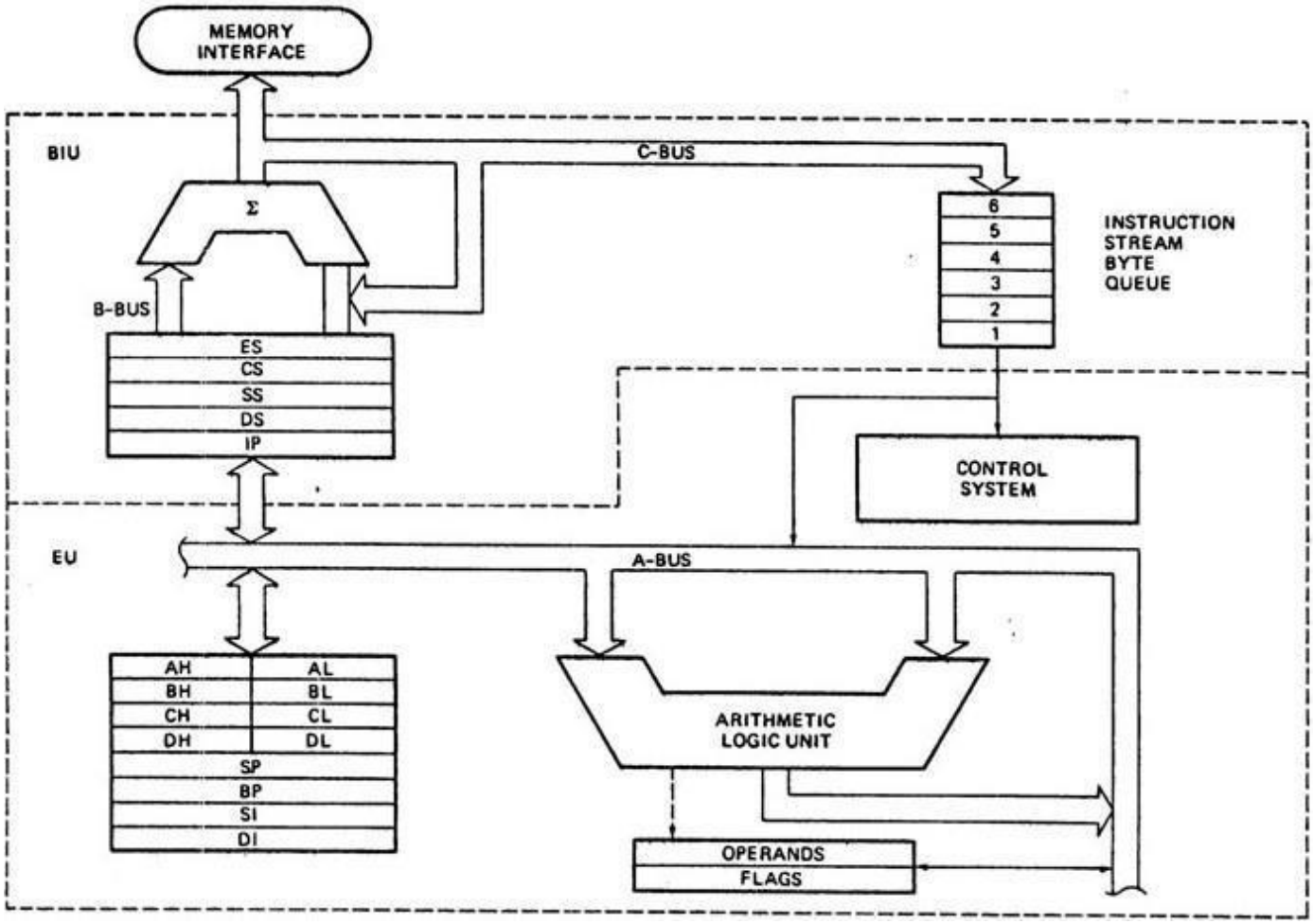
In case of stack addressing mode, the instruction knows the **topmost data should be the operand**. If the instruction is a unary instruction then it will select the topmost data as the operand and if the instruction is a binary instruction then it will select the topmost two data as the operands from the top of the stack.



These are the basic and primary addressing modes in computer architecture. Apart from these modes, we have some other addressing modes in computer architecture including auto-increment, auto-decrement mode. But above mentioned are the most important addressing modes in computer architecture or computer organisation.

8086 Micro-Processor Architecture:

The Block diagram for the internal architecture of 8086 is as follows:-



- In 8086 CPU is divided into two independent functional parts BIU and EU.
- Dividing the work between these two units' speeds up the processing.

BIU (Bus Interface Unit)

Components of BIU

- Instruction queue
  It holds the instruction bytes of the next instruction to be executed by EU
- Segment Registers
  Four 16-bit register that provides powerful memory management mechanism
  ES (extra segment), CS (code segment), SS (stack segment) , DS (data segment). The size of each register is 64kb.
- Instruction pointer (IP)
  Register that holds 16-bit address or offset of next code byte within code segment
- Address Generation and bus control
  Generation of 20-bit physical address

Task carried out by BIU

- Fetch instruction from memory

- Read/ Write instruction from / to the memory
- Input/ Output (I/O) of data from / to peripheral ports
- Write the data to memory.
- Address generation for memory reference
- Queuing of instruction (The instruction bytes are transferred to the instruction queue)
- Thus, BUI handles all transfer of data and address on the buses for Execution unit.
- BIU works in synchronous with machine cycles

EU (Execution Unit)

Components of EU

- ALU (Arithmetic logic Unit)
  Contains 16-bit ALU, that performs add, subtract, increment, decrement, compliment, shift binary numbers, AND, OR, XOR etc.
- CU (Control Unit)
  Directs internal operation
- Flag Register
  16-bit flag register. EU contains 9 active flags
- General Purpose Registers (GPR)
  EU has 4 general purpose 16-bit register i.e. AX, BX, CX, DX each register is the combination of two 8-bit register AH, AL, BH, BL, CH, CL, DH, DL where „L" means Lower byte and „H" means higher byte.
- Index Register
  16-bit Register is SI (source index) and DI (destination index). Both the register are used for string related operation and for moving block of memory from one location to the other.
- Pointers
  16-bit Register. i.e, SP (stack pointer), BP (base pointer)
  BP: is used when we need to pass parameter through stack
  SP: It always points to the top of the stack. Used for sequential access of stack segment.
- Decoder (instruction decoder)
  Translates the instruction fetched from into series of action which EU carries out

Task carried out by EU

- Decodes the instruction
- It executes instructions (executes decoded instructions)
- Tells BIU from where to fetch the instruction
- Decodes instruction (decode the fetched instruction)
- EU takes care of performing operation on the data
- EU is also known as execution heart of the processor

General Purpose Register

AX: - Accumulator register consists of two 8-bit registers AL and AH, which can be combined together and used as a 16-bit register AX.

- AX works as an intermediate register in memory and I/O operation.
- Accumulator is used for the instruction such as MUL and DIV.

BX: - Base register consists of two 8-bit registers BL and BH, which can be combined together and used as a 16-bit register BX.

- BX register usually contains a data pointer used for based, based indexed or register indirect addressing.

CX: - Count register consists of two 8-bit registers CL and CH, which can be combined together and used as a 16-bit register CX. Count register can be used in Loop, shift/rotate instructions and as a counter in string manipulation.

DX: - Data register can be used together with AX register to execute MUL and DIV instruction.

- Data register can be used as a port number in I/O operations.

Segment Register

Types of Segment registers are as follows:-

- **Code Segment (CS):** The CS register is used for addressing a memory location in the Code Segment of the memory, where the executable program is stored.
- **Data Segment (DS):** The DS contains most data used by program. Data are accessed in the Data Segment by an offset address or the content of other register that holds the offset address.
- **Stack Segment (SS):** SS defined the area of memory used for the stack.
- **Extra Segment (ES):** ES is additional data segment that is used by some of the string to hold the destination data

Pointer Registers

The pointers IP, BP, SP usually contain offsets within the code, data and stack segments respectively.

- **Stack Pointer (SP):** SP is a 16-bit register pointing to program stack in stack segment.
- **Base Pointer (BP):** BP is a 16-bit register pointing to data in stack segment. BP register is usually used for based, based indexed or register indirect addressing.
- **Instruction Pointer (IP):** IP is a 16-bit register pointing to next instruction to be executed.

Index registers

The Index Registers are as follows:-

- **Source Index (SI):** SI is a 16-bit register used for indexed, based indexed and register indirect addressing, as well as a source data addresses in string manipulation instructions.
- **Destination Index (DI):** DI is a 16-bit register. DI is used for indexed, based indexed and register indirect addressing, as well as a destination data addresses in string manipulation instructions.

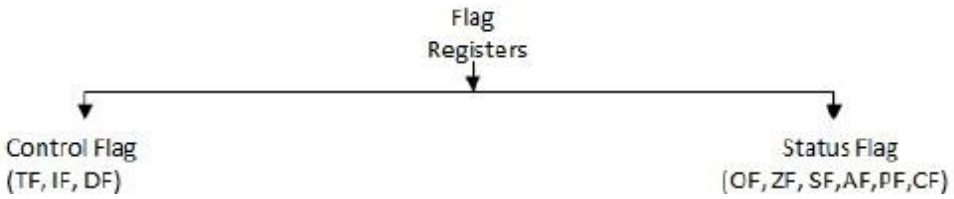
Flag Registers

- The 16-bit flag register of 8086 contains 9 active flags (six conditional & 3 control flags), other 7 flags are undefined.
- Status Flags: It indicates certain condition that arises during the execution. They are controlled by the processor.
- Control Flags: It controls certain operations of the processor. They are deliberately set/ reset by the user.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U	U	U	U	OF	DF	IF	TF	SF	ZF	U	AF	U	PF	U	CF

U- Undefined
TF- Trap Flag
IF- Interrupt Flag
DF- Direction Flag
OF- Overflow Flag
CF- Carry Flag
PF- Parity Flag
AF- Auxiliary Flag
ZF- Zero Flag
SF- Sign Flag

The Flag Registers are classified as follows :-



Control Flags

Control flags are set or reset deliberately to control the operations of the execution unit.

1. **Trap Flag (TF):**
   - It is used for single step control.
   - It allows user to execute one instruction of a program at a time for debugging.
   - When trap flag is set, program can be run in single step mode.
2. **Interrupt Flag (IF):**
   - It is an interrupt enable/disable flag.
   - If it is set, the mask able interrupt of 8086 is enabled and if it is reset, the interrupt is disabled.
   - It can be set by executing instruction sit and can be cleared by executing CLI instruction.
3. **Direction Flag (DF):**
   - It is used in string operation.
   - If it is set, string bytes are accessed from higher memory address to lower memory address.
   - When it is reset, the string bytes are accessed from lower memory address to higher memory address.

Status Flag

1. **Carry Flag (CF):** This flag indicates an overflow condition for unsigned integer arithmetic. It is also used in multiple-precision arithmetic.
2. **Auxiliary Flag (AF):** If an operation performed in ALU generates a carry/barrow from lower nibble (i.e. D0 – D3) to upper nibble (i.e. D4 – D7), the AF flag is set i.e. carry given by D3 bit to D4 is AF flag. This is not a general-purpose flag, it is used internally by the processor to perform Binary to BCD conversion.
3. **Parity Flag (PF):** This flag is used to indicate the parity of result. If lower order 8-bits of the result contains even number of 1‟s, the Parity Flag is set and for odd number of 1‟s, the Parity Flag is reset.

4. **Zero Flag (ZF):** It is set; if the result of arithmetic or logical operation is zero else it is reset.
5. **Sign Flag (SF):** In sign magnitude format the sign of number is indicated by MSB bit. If the result of operation is negative, sign flag is set.
6. **Overflow Flag (OF):** It occurs when signed numbers are added or subtracted. OF=1 indicates that the result has exceeded the capacity of machine.